

# BRIMS Technology Stack

Barangay Resident Information Management System

Generated: May 15, 2026

BRIMS is a full-stack web application for barangay staff and residents to manage profiles, households, certificates, and requests. This document summarizes the technologies used in the project.

## Core Platform

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| Frontend        | Angular 21 (standalone components, lazy routes), TypeScript 5.9, SCSS |
| Runtime         | Node.js v18+ (development, build tooling, and backend)                |
| Package manager | npm                                                                   |

## Frontend (Angular App)

- Angular ecosystem: Core, Router, Forms, Animations, Angular CDK
- Reactive programming: RxJS 7.8
- State: Angular signals (project conventions)
- Database (client): Firebase 12 + AngularFire with Cloud Firestore as primary datastore
- Auth / passwords: bcryptjs (hashed passwords stored in Firestore)
- QR scanning: @zxing/ngx-scanner (camera-based QR codes)
- Charts / reports: Chart.js + ng2-charts
- Maps: Leaflet (household and geographic views)
- PDF / export: jsPDF, html2canvas
- UI / UX: SweetAlert2, GSAP (animations), Three.js (3D where used)
- Optional demo API: json-server (mock REST from server/db.json)

## Backend (backend/)

- Node.js + Express 4
- CORS, dotenv, multer (file uploads)
- SMS: Twilio
- Email: Nodemailer (SMTP)
- Development: nodemon

Default URLs: Angular dev server <http://localhost:4200>; notification API <http://localhost:4000>.

## Data & Configuration

- Primary database: Google Firebase / Cloud Firestore
- Runtime config: src/assets/config.json (Firebase + API URL; gitignored)
- Seeding: firebase-admin + scripts/seed-firestore.js for demo data
- Abstraction: IDatabaseService with Firestore (active), local storage, and JSON Server implementations

## Architecture Patterns

- Route guards for authentication and role-based access (Admin, Staff, Resident)
- Standalone components and lazy-loaded feature routes
- Swappable data layer via dependency injection (DATABASE\_SERVICE)

## Not Used in This Repository

- React or Next.js (Angular-only frontend)
- Docker / Kubernetes deployment configs in the repo root
- Azure Pipelines YAML in the scanned project root

**Summary: Angular 21 + TypeScript + SCSS on the frontend; Firestore via Firebase/AngularFire for data; a small Express backend for SMS/email (Twilio + Nodemailer); plus Leaflet, Chart.js, and ZXing for maps, charts, and QR scanning.**